

ATM SYSTEM

PROJECT REPORT

VITYARTHI PROJECT

PDF

Programming Language: Python

PDF+ 1

Academic Year: 2026

PDF

SUBMITTED BY: DHRUV GOYAL

REGISTRATION NO.: 26BCE10208

1. Introduction

For my VITYarthi project, I built an **ATM System** using Python. I selected this project because an Automated Teller Machine (ATM) is a practical, real-life financial application where fundamental programming concepts like conditional logic, iteration, dictionaries, lists, and security validation work together.

PDF+ 3

Option / Function	What it does
Account & PIN Authentication	Asks for Account Number, validates if it exists, and grants access after verifying the correct 4-digit PIN (max 3 attempts allowed).
Check Balance	Displays the current available balance of the logged-in account holder.

Deposit Money

Takes input amount, checks if greater than zero, updates account balance, and logs transaction.

PY

Withdraw Money

Asks for amount, verifies sufficient balance, deducts money from account, and records transaction history.

PY

Change PIN

Validates current PIN, prompts for new PIN twice, and updates PIN upon match.

PY

Mini Statement

Displays a list of all recent deposits and withdrawals performed during the current session along with final balance.

PY

Logout & Exit

Logout ends the active account session, while Exit safely closes the ATM system.

PY

The program runs in the interactive terminal and offers a numbered main menu options (1–7). While making this project, I used core Python data structures such as nested dictionaries for account management, lists for transaction tracking, `while` loops for continuous operation, `if-else` conditionals for decision-making, and string formatting.

PDF+ 3

2. Problem Statement

In daily financial operations, managing user accounts, verifying security PINs, maintaining transaction trails, and ensuring real-time balance updates are essential. Manual handling or unverified access can lead to security flaws and calculation errors.

PDF+ 2

The purpose of making this project is to combine all basic banking operations—authentication, cash deposit, cash withdrawal, PIN management, and transaction reporting—into a single console application that simulates real-world ATM behavior safely and accurately.

PDF+ 1

3. Objectives

a. To construct a fully functional Python-based console project addressing a real-life banking system.

PDF+ 1

b. To implement multi-account security validation using Account Numbers and 4-digit PINs.

PY

c. To restrict unauthorized access by blocking accounts after 3 consecutive wrong PIN attempts.

PY

d. To process deposits and withdrawals with real-time balance calculations and boundary checks.

PY

e. To maintain an in-memory audit log (`transactions` list) for generating mini statements.

PY

4. Functional Requirements

The program operates sequentially through a terminal interface:

PDF+ 1

1. User enters their Account Number.
PY
2. System checks account existence in the `accounts` dictionary.
PY
3. User enters PIN (up to 3 attempts allowed). If incorrect 3 times, account gets locked.
PY
4. Upon successful login, the main menu displays options 1 through 7.
PY

5. User selects an option, performs transactions or changes PIN, and can view a mini statement before logging out or exiting.
PY

5. Non-Functional Requirements

- a. **Usability:** Simple navigation through numbered menu options and intuitive console outputs.

PDF+ 1

- b. **Reliability:** Validates sufficient account balance before completing withdrawals to prevent negative balances.

PY

- c. **Security:** Blocks account access after 3 failed PIN attempts to simulate security compliance.

PY

- d. **Maintainability:** Clean separation of account storage, transaction logging, and menu control flow.

PDF+ 1

- e. **Performance:** Instant execution for terminal user requests without noticeable delay.

PDF

- f. **Resource Efficiency:** Developed purely using core Python built-ins without requiring external packages.

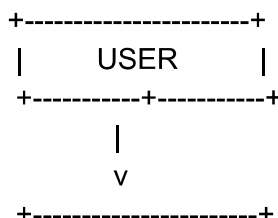
PDF+ 1

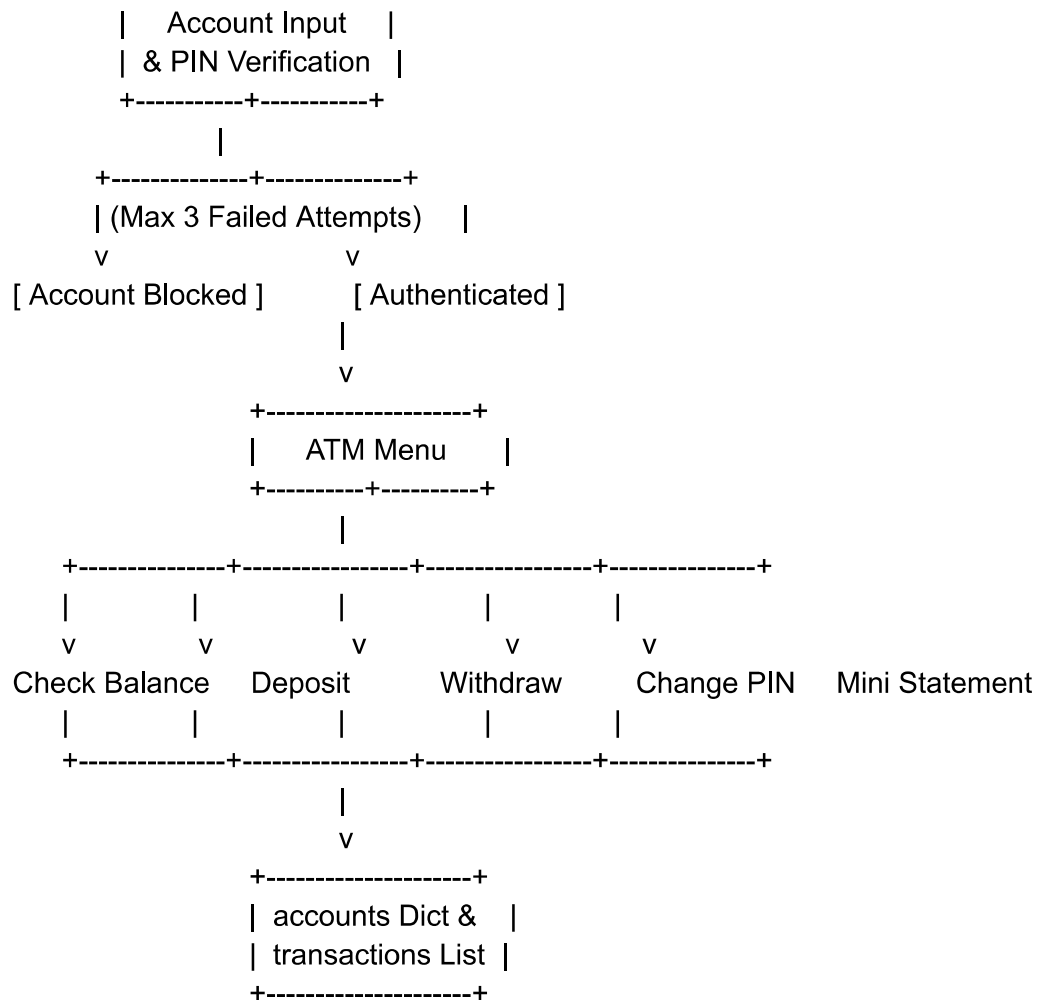
6. System Architecture

The user interacts with `ATM.py`. The program verifies inputs against `accounts` (dictionary data store) and records activities inside the `transactions` array.

PDF+ 2

Plaintext





7. Design Decisions

- Dictionaries for Account Storage:** Nested dictionaries (`accounts`) mapping `account_number` to `{"name", "pin", "balance"}` were selected because key-value lookup is extremely efficient ($O(1)$ complexity).
 PY
- List for Transaction Logging:** A `transactions` list was used to record formatted string entries (`+ Deposited: ₹X / - Withdrawn: ₹X`) sequentially as they occur during the session.
 PY
- Attempt Counter Loop:** A `while attempts < 3` loop ensures security enforcement, terminating execution if the limit is reached.
 PY

- **Input Boundaries:** Withdrawal amounts are evaluated against `amount <= account["balance"]` to guarantee financial integrity.
PY

8. Implementation Details

a. Data Initialization

Account details are stored in the `accounts` dictionary.

```
PY
Python
accounts = {
    1001: {"name": "Dhruv", "pin": 3252, "balance": 5000},
    1002: {"name": "Harsh", "pin": 3252, "balance": 7500},
    ...
}
transactions = []
```[cite: 2]
```

### ##### b. Authentication Logic

Verifies Account Number existence and checks input PIN up to 3 times[cite: 2].

```
```python
account_number = int(input("Enter your account number: "))
if account_number in accounts:
    account = accounts[account_number]
else:
    print("Invalid account number")
    exit()
```[cite: 2]
```

### ##### c. Core Menu Options (`while` running:)

- **Choice 1 (Check Balance):** Displays `account["balance"]`[cite: 2].
- **Choice 2 (Deposit):** Checks `amount > 0`, updates balance, and appends to `transactions`[cite: 2].
- **Choice 3 (Withdraw):** Validates `0 < amount <= balance`, deducts amount, and appends to `transactions`[cite: 2].
- **Choice 4 (Change PIN):** Verifies current PIN, matches new PIN confirmation, and updates `account["pin"]`[cite: 2].
- **Choice 5 (Mini Statement):** Loops through `transactions` list using `enumerate()` and prints summary[cite: 2].
- **Choice 6 & 7 (Logout / Exit):** Terminates session loop cleanly[cite: 2].

---

### ### 9. Screenshots / Results

Output previews generated by execution of the terminal program[cite: 1, 2]:

#### #### Screen 1: Login & Authentication

```
```text
```

Enter your account number: 1001

Enter your PIN: 3252

Welcome Dhruv !

PIN is correct

```
=====
```

ATM SYSTEM

```
=====
```

Account Holder: Dhruv

```
=====
```

MAIN MENU

```
-----
```

1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Change PIN
5. Mini Statement
6. Logout
7. EXIT

```
-----
```

Enter your choice: 1

Your balance is: ₹ 5000

```
```[cite: 2]
```

#### #### Screen 2: Deposit & Withdrawal Operations

```
```text
```

Enter your choice: 2

Enter amount to deposit: ₹ 1500

Amount deposited successfully

Your new balance is: ₹ 6500.0

Enter your choice: 3

Enter amount to withdraw: ₹ 2000

Please collect your cash
Your remaining balance is: ₹ 4500.0
```[cite: 2]

#### ##### Screen 3: Mini Statement

```text

Enter your choice: 5

=====

MINI STATEMENT

=====

Account Holder: Dhruv

1 . + Deposited: ₹1500.00

2 . - Withdrawn: ₹2000.00

Final Balance: ₹ 4500.00

=====

```[cite: 2]

#### ##### Screen 4: Security Account Lock Simulation

```text

Enter your account number: 1001

Enter your PIN: 1111

PIN is incorrect

Enter your PIN: 2222

PIN is incorrect

Enter your PIN: 3333

PIN is incorrect

Too many attempts. Your account is blocked

```[cite: 2]

---

#### ### 10. Testing Approach

I tested the program across multiple test cases covering both expected and invalid inputs[cite: 1, 2]:

| Test Case | Expected Behaviour | Result |
|-----------|--------------------|--------|
|-----------|--------------------|--------|

|      |      |      |
|------|------|------|
| :--- | :--- | :--- |
|------|------|------|

|                            |                                                                    |
|----------------------------|--------------------------------------------------------------------|
| **Invalid Account Number** | Program prints "Invalid account number" and exits safely[cite: 2]. |
|----------------------------|--------------------------------------------------------------------|

|                 |
|-----------------|
| Passed[cite: 2] |
|-----------------|

| **\*\*Incorrect PIN (3 Attempts)\*\*** | System prints "Too many attempts. Your account is blocked"[cite: 2]. | Passed[cite: 2] |  
| **\*\*Withdrawal > Balance\*\*** | Program displays "Insufficient balance" and cancels withdrawal[cite: 2]. | Passed[cite: 2] |  
| **\*\*Negative Deposit Amount\*\*** | Displays "Invalid amount" without altering balance[cite: 2]. | Passed[cite: 2] |  
| **\*\*Mismatched New PIN\*\*** | Displays "New PINs do not match" and retains old PIN[cite: 2]. | Passed[cite: 2] |

---

#### ### 11. Challenges Faced

- **\*\*Limiting PIN Retry Attempts:\*\*** Ensuring the user gets exactly 3 attempts before locking out required structuring a `while attempts < 3` loop with conditional breaks[cite: 2].
- **\*\*Transaction History Tracking:\*\*** Formatting string representations of transactions dynamically into a list so that `Mini Statement` prints them indexed starting from 1 using `enumerate()` [cite: 2].
- **\*\*Boundary Checks on Balance:\*\*** Ensuring withdrawals do not exceed account balance or accept negative values[cite: 2].

---

#### ### 12. Learnings & Key Takeaways

- Understood how fundamental Python data structures like dictionaries and lists store structured application state in real-time[cite: 1, 2].
- Practical implementation of input validation, state modification, and loop controls[cite: 1, 2].
- Experience in structuring professional documentation and presenting software projects on GitHub[cite: 1].

---

#### ### 13. Future Enhancements

- a. File handling or database integration (SQLite) for permanent balance persistence across sessions[cite: 1].
- b. Multi-user session persistence after logout without restarting the program[cite: 2].
- c. Support for Fund Transfer between different account numbers[cite: 2].
- d. Graphical User Interface (GUI) built using Python Tkinter[cite: 1].

---

#### ### 14. References

- VITyarthi Python Essentials course
- Python 3 Documentation (`docs.python.org`)
- Visual Studio Code Documentation]

- GitHub Documentation
- Project Source File: `ATM.org.py`

**END OF REPORT**